Design and Implementation of a 4-Bit Processor using Verilog HDL

Nov 4, 2026 · @Nikita

Project Report · Department of Electronics and Telecommunication Engineering, Jaihind College of Engineering, Kuran, Pune

Abstract

This project presents the design and simulation of a 4-bit single-cycle processor described in Verilog HDL. The processor has a 4-bit data path, a 4-bit program counter, a 16 × 12-bit instruction memory, a 4 × 4-bit register file, a 16 × 4-bit data memory and a 4-bit ALU with six operations. Each 12-bit instruction carries a 4-bit opcode, two 2-bit register fields and a 4-bit immediate value. The design is built from seven modules and is verified with a testbench in Icarus Verilog on EDA Playground. A five-instruction test program (ADD, SUB, AND, OR and HALT) is fetched in order, the program counter steps from 0 to 4, and the processor halts at 45 ns.

Keywords: 4-bit processor, Verilog HDL, ALU, register file, control unit, single-cycle architecture, RTL simulation

List of Abbreviations

Abbreviation	Meaning
ALU	Arithmetic Logic Unit
ASIC	Application-Specific Integrated Circuit
CU	Control Unit
DM	Data Memory
FPGA	Field-Programmable Gate Array
HDL	Hardware Description Language
IM	Instruction Memory
MUX	Multiplexer (selector)
PC	Program Counter
RF	Register File
RTL	Register-Transfer Level

Abbreviation	Meaning
VCD	Value Change Dump

1. Introduction

1.1 Background

A processor fetches instructions from memory, decodes them and executes them on data held in registers and memory. Even a very small processor contains the same basic parts as a large one: a program counter, instruction memory, a control unit, a register file, an arithmetic logic unit (ALU) and data memory. Designing such a processor in a hardware description language is one of the most effective ways to understand computer organisation and digital design.

Verilog HDL describes hardware at the register-transfer level (RTL). The same code can be simulated to check behaviour and later synthesised for an FPGA or ASIC.

1.2 Objectives

- To design a 4-bit single-cycle processor with a 12-bit instruction format.
- To describe each block (ALU, register file, control unit, memories, program counter) as a separate Verilog module.
- To verify the complete processor with a testbench and a test program.
- To document the architecture, instruction set, code and simulation results.

1.3 Scope

The processor executes one instruction per clock cycle. It supports arithmetic and logic operations with an immediate value, load, store, register move and halt. Branching, interrupts and a stack are outside the scope of this version.

1.4 Organisation of the Report

Section 2 describes the tools and the design flow. Section 3 explains the architecture and the data flow, Section 4 defines the instruction set, and Section 5 describes the control unit and the execution of each instruction class. Section 6 lists the Verilog code of all seven modules, Section 7 describes the testbench, and Section 8 presents the test program, the console output, the waveform and a verification summary. Sections 9 to 11 cover the advantages, limitations, applications, future scope and conclusion.

2. Tools and Technologies

Item	Description
Language	Verilog HDL, compiled with the SystemVerilog-2012 option
Simulator	Icarus Verilog (`iverilog -Wall -g2012` , run with `vvp`)
Environment	EDA Playground (online simulation)
Waveform output	VCD file written with `$dumpfile` and `$dumpvars`

2.1 Design Flow

The project followed the usual RTL design flow:

1. **Specification:** decide the word length, instruction format, registers, memories and the instruction set.
2. **Architecture:** divide the processor into blocks and define the signals between them.
3. **RTL coding:** write each block as a Verilog module and connect the modules in a top-level module.
4. **Testbench:** write a testbench that supplies the clock and reset and observes the outputs.
5. **Simulation:** compile with Icarus Verilog, run the simulation and check the console output and the waveform.
6. **Analysis:** compare the observed behaviour with the expected behaviour and note the limitations for future work.

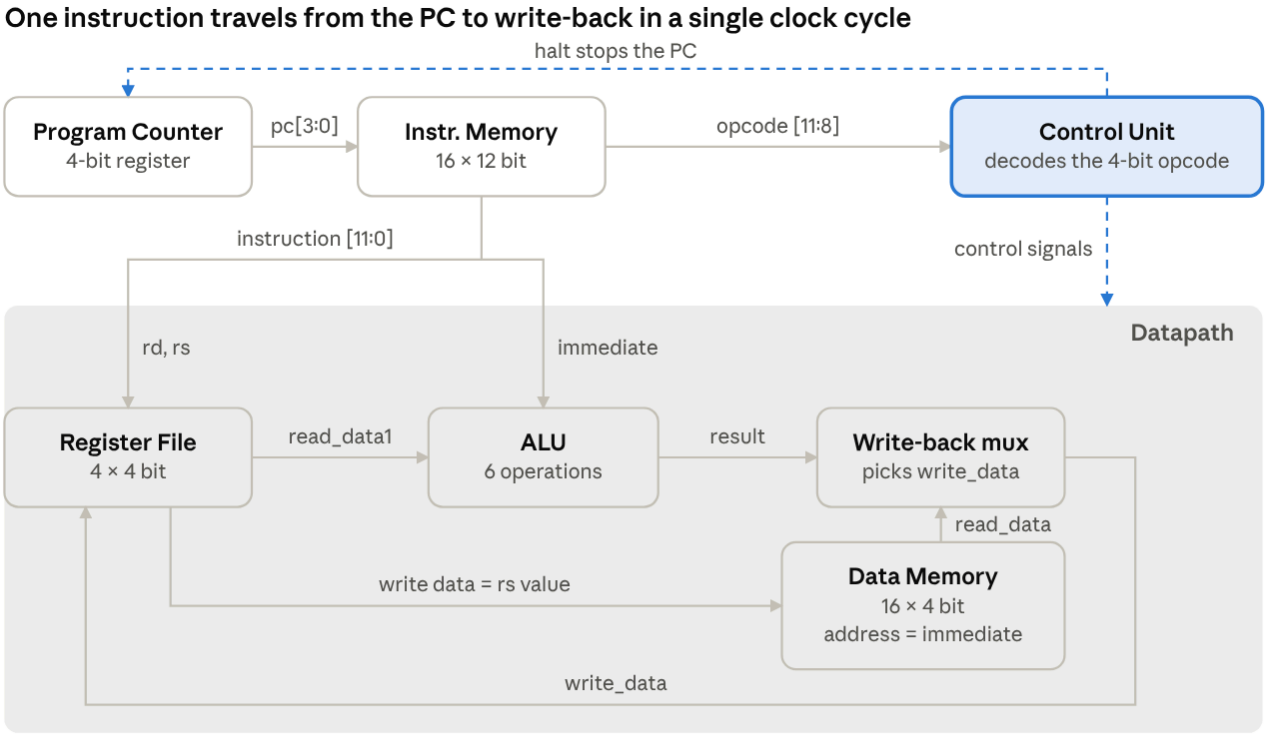
2.2 Modelling Style

Combinational logic (the ALU, the control unit, the memory reads and the write-back selection) is written with `always @(*)` blocks or continuous `assign` statements. Sequential logic (the program counter, the register file and the data-memory writes) is written with `always @(posedge clk or posedge reset)` blocks that use non-blocking assignments, so that all registers update together on the clock edge. This separation keeps the code synthesisable and avoids race conditions between blocks.

3. System Architecture

The processor is a single-cycle design: the instruction at the address held in the program counter is read, decoded and executed within one clock period, and the program counter then moves to the next address. The instruction memory is read without a clock (asynchronously), while the register file and the data memory are written on the rising clock edge.

Block	Size	Function
Program Counter	4 bit	Holds the address of the current instruction, increments on every clock and stops when HALT is decoded; reset to 0
Instruction Memory	16 × 12 bit	Stores the program; read asynchronously
Control Unit	Combinational	Decodes the 4-bit opcode into reg_write, mem_read, mem_write, alu_src, halt and alu_sel
Register File	4 × 4 bit	Two read ports and one write port; synchronous write and asynchronous reset
ALU	4 bit	ADD, SUB, AND, OR, XOR and NOT with Zero and Carry outputs
Data Memory	16 × 4 bit	Read when mem_read is high, written when mem_write is high; address comes from the immediate field
Write-back selector	4 bit	Chooses memory data (LOAD), register data (MOV) or the ALU result as the value written to the register file



block diagram · 7 blocks, instruction path and datapath

Figure 1: Block diagram of the 4-bit processor

3.1 Data Flow

The 12-bit instruction is split into four fields: opcode (bits 11 to 8), destination register rd (bits 7 to 6), source register rs (bits 5 to 4) and immediate (bits 3 to 0). The register file is read with rs, and the ALU combines that value with the immediate. The result, the memory data or the register data is selected according to the opcode and written back into the register rd. For STORE, the value of rs is written into the data memory at the address given by the immediate.

3.2 Single-Cycle Operation

In a single-cycle processor every instruction completes in exactly one clock cycle, so the clock period must be long enough for the slowest path: instruction fetch, decode, register read, ALU operation (or memory access) and write-back. The advantage is simplicity, because no instruction register, state machine or intermediate storage is needed. The disadvantage is that the clock cannot be faster than the longest instruction path. For this small 4-bit design that trade-off is acceptable.

3.3 Clocking and Reset

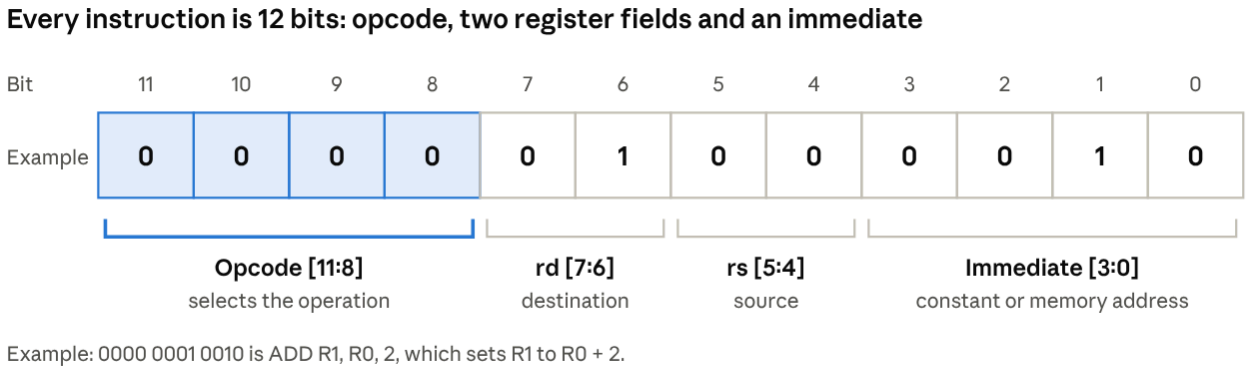
The program counter, the register file and the data memory respond to the rising edge of the clock. Reset is asynchronous and active high: when reset goes high the program counter returns to address 0, all four registers are cleared and all sixteen data-memory words are cleared. In the testbench the clock period is 10 ns (rising edges at 5, 15, 25 ns and so on) and reset is released at 10 ns, so the first instruction is committed at the 15 ns edge.

3.4 Data Representation

All data values are 4-bit unsigned numbers in the range 0 to 15. Arithmetic is modulo 16: for example 0 − 4 gives 1100 (12), and 9 + 8 gives 0001 with the carry output set. The immediate field is also 4 bits wide, so constants and data-memory addresses are limited to 0 to 15, which matches the 16 locations of the data memory.

4. Instruction Set Architecture

Every instruction is 12 bits wide.



instruction format · 12-bit word, four fields

Figure 2: Instruction format

Mnemonics are used in this report for readability; the hardware only sees the opcode bits.

Opcode	Mnemonic	Operation
0000	ADD rd, rs, imm	R[rd] ← R[rs] + imm
0001	SUB rd, rs, imm	R[rd] ← R[rs] − imm
0010	AND rd, rs, imm	R[rd] ← R[rs] AND imm
0011	OR rd, rs, imm	R[rd] ← R[rs] OR imm

Opcode	Mnemonic	Operation
0100	XOR rd, rs, imm	R[rd] ← R[rs] XOR imm
0101	NOT rd, rs	R[rd] ← NOT R[rs] (immediate ignored)
0110	LOAD rd, imm	R[rd] ← DMEM[imm]
0111	STORE rs, imm	DMEM[imm] ← R[rs]
1000	MOV rd, rs	R[rd] ← R[rs]
1111	HALT	Stop the program counter and assert halted
Others	–	No operation (no register or memory write)

4.1 Addressing Modes

Mode	Used by	Description
Register	ADD, SUB, AND, OR, XOR, NOT, MOV, STORE	The operand is read from the register selected by rs
Immediate	ADD, SUB, AND, OR, XOR	The second ALU operand is the 4-bit immediate field
Direct (memory)	LOAD, STORE	The immediate field is used as the address of the data memory

4.2 Encoding Examples

The table shows how some instructions are encoded. Fields that an instruction does not use are written as 0.

Instruction	Opcode	rd	rs	Immediate	Machine code	Hex
ADD R1, R0, 2	0000	01	00	0010	0000 0001 0010	012
XOR R3, R1, 12	0100	11	01	1100	0100 1101 1100	4DC
NOT R0, R2	0101	00	10	0000	0101 0010 0000	520
LOAD R2, 3	0110	10	00	0011	0110 1000 0011	683
STORE R1, 3	0111	00	01	0011	0111 0001 0011	713
MOV R3, R0	1000	11	00	0000	1000 1100 0000	8C0
HALT	1111	00	00	0000	1111 0000 0000	F00

5. Control Unit and Execution Flow

The control unit is purely combinational. It reads the opcode and drives the control signals listed below. When HALT is decoded, the halt signal stops the program counter, blocks register writes and is passed to the output as `halted`.

Opcode	Instruction	reg_write	mem_read	mem_write	alu_src	halt	alu_sel
0000	ADD	1	0	0	0	0	000
0001	SUB	1	0	0	0	0	001
0010	AND	1	0	0	0	0	010
0011	OR	1	0	0	0	0	011
0100	XOR	1	0	0	0	0	100
0101	NOT	1	0	0	0	0	101
0110	LOAD	1	1	0	1	0	000
0111	STORE	0	0	1	1	0	000
1000	MOV	1	0	0	0	0	000
1111	HALT	0	0	0	0	1	000

Figure 3: Execution flow of one instruction

5.1 Control Signal Functions

Signal	Function
reg_write	Allows the register file to store write_data in register rd at the next rising clock edge (gated with the inverse of halt in the top-level module)
mem_read	Makes the data memory output the word at the address given by the immediate; otherwise its output is 0
mem_write	Writes the value of register rs into the data memory at the next rising clock edge
alu_src	Marks the instructions that use the immediate as an address or constant (LOAD and STORE); it is generated but not yet used in the datapath
halt	Stops the program counter and is passed to the testbench as halted

Signal	Function
alu_sel	3-bit code that selects the ALU operation: 000 ADD, 001 SUB, 010 AND, 011 OR, 100 XOR, 101 NOT

5.2 Execution of Each Instruction Class

Arithmetic and logic (ADD, SUB, AND, OR, XOR, NOT). The register file outputs R[rs] on read_data1. The ALU combines it with the immediate according to alu_sel, and the result is selected by the write-back mux and stored in R[rd] at the clock edge. NOT uses only the register value.

LOAD. The immediate is applied as the address of the data memory, mem_read is high, and the memory output is selected by the write-back mux and stored in R[rd].

STORE. R[rs] is placed on the data input of the data memory, the immediate is the address and mem_write is high, so the word is written at the clock edge. No register is changed.

MOV. The write-back mux passes read_data1 (R[rs]) directly, so R[rd] receives a copy of R[rs] without using the ALU result.

HALT. The control unit sets halt. The program counter stops incrementing, register writes are blocked and halted goes high, which tells the testbench that the program has finished.

Unassigned opcodes. Opcodes 1001 to 1110 are not defined. The control unit drives all signals low for them, so they behave as no-operation instructions.

6. Verilog HDL Implementation

The design is written in a single file, Processor.v, with seven modules. The listings below show the complete code; only the indentation has been adjusted for readability.

6.1 ALU

The ALU performs six operations selected by the 3-bit ALU_Sel input. The Zero output is high when the result is zero, and Carry is set only by addition.

```
module alu(
    input  [3:0] A,
    input  [3:0] B,
    input  [2:0] ALU_Sel,
    output reg [3:0] Result,
    output reg Zero,
    output reg Carry
```

```
);

    always @(*) begin
        Carry  = 0;
        Result = 0;

        case (ALU_Sel)
            3'b000: {Carry, Result} = {1'b0, A} + {1'b0, B};
            3'b001: Result = A - B;
            3'b010: Result = A & B;
            3'b011: Result = A | B;
            3'b100: Result = A ^ B;
            3'b101: Result = ~A;
            default: Result = 0;
        endcase

        Zero = (Result == 0);
    end

endmodule
```

6.2 Register File

Four 4-bit registers are cleared on reset. Two read ports are provided, and one register is written on the clock edge when reg_write is high.

```
module register_file(
    input clk,
    input reset,
    input reg_write,
    input [1:0] read_addr1,
    input [1:0] read_addr2,
    input [1:0] write_addr,
    input [3:0] write_data,
    output [3:0] read_data1,
    output [3:0] read_data2
);

    reg [3:0] registers [0:3];
    integer i;

    always @(posedge clk or posedge reset) begin
        if (reset) begin
            for (i=0; i<4; i=i+1)
                registers[i] <= 0;
        end
```

```
        else if (reg_write) begin
            registers[write_addr] <= write_data;
        end
    end

    assign read_data1 = registers[read_addr1];
    assign read_data2 = registers[read_addr2];

endmodule
```

6.3 Control Unit

```
module control_unit(
    input [3:0] opcode,
    output reg reg_write,
    output reg mem_read,
    output reg mem_write,
    output reg alu_src,
    output reg halt,
    output reg [2:0] alu_sel
);

    always @(*) begin

        reg_write = 0;
        mem_read  = 0;
        mem_write = 0;
        alu_src   = 0;
        halt      = 0;
        alu_sel   = 0;

        case (opcode)

            4'b0000: begin
                reg_write = 1;
                alu_sel = 3'b000;
            end

            4'b0001: begin
                reg_write = 1;
                alu_sel = 3'b001;
            end

            4'b0010: begin
                reg_write = 1;
                alu_sel = 3'b010;
```

```
            end

            4'b0011: begin
                reg_write = 1;
                alu_sel = 3'b011;
            end

            4'b0100: begin
                reg_write = 1;
                alu_sel = 3'b100;
            end

            4'b0101: begin
                reg_write = 1;
                alu_sel = 3'b101;
            end

            4'b0110: begin
                reg_write = 1;
                mem_read = 1;
                alu_src = 1;
            end

            4'b0111: begin
                mem_write = 1;
                alu_src = 1;
            end

            4'b1000: begin
                reg_write = 1;
            end

            4'b1111: begin
                halt = 1;
            end

            default: begin
                reg_write = 0;
            end

        endcase

    end

endmodule
```

6.4 Instruction Memory

The memory holds 16 instructions of 12 bits. All locations are first cleared, and then the five test instructions are loaded.

```
module instruction_memory(
    input [3:0] address,
    output reg [11:0] instruction
);

    reg [11:0] memory [0:15];
    integer i;

    initial begin

        for (i=0; i<16; i=i+1)
            memory[i] = 12'b000000000000;

        memory[0] = 12'b000000010010;
        memory[1] = 12'b000100110100;
        memory[2] = 12'b001001010110;
        memory[3] = 12'b001101110000;
        memory[4] = 12'b111100000000;

    end

    always @(*) begin
        instruction = memory[address];
    end

endmodule
```

6.5 Data Memory

```
module data_memory(
    input clk,
    input reset,
    input mem_read,
    input mem_write,
    input [3:0] address,
    input [3:0] write_data,
    output reg [3:0] read_data
);

    reg [3:0] memory [0:15];
    integer i;
```

```
    always @(posedge clk or posedge reset) begin

        if (reset) begin
            for (i=0; i<16; i=i+1)
                memory[i] <= 0;
        end

        else if (mem_write) begin
            memory[address] <= write_data;
        end

    end

    always @(*) begin
        if (mem_read)
            read_data = memory[address];
        else
            read_data = 0;
    end

endmodule
```

6.6 Program Counter

```
module program_counter(
    input clk,
    input reset,
    input enable,
    input halt,
    output reg [3:0] pc
);

    always @(posedge clk or posedge reset) begin

        if (reset)
            pc <= 0;

        else if (enable && !halt)
            pc <= pc + 1'b1;

    end

endmodule
```

6.7 Top-Level Processor Module

The top-level module connects all blocks. The write-back value is selected by the opcode: memory data for LOAD, register data for MOV and the ALU result for everything else.

```
module processor(
    input clk,
    input reset,
    output [3:0] pc_out,
    output [11:0] instruction_out,
    output halted
);

    wire [3:0] pc;

    program_counter PC(
        .clk(clk),
        .reset(reset),
        .enable(1'b1),
        .halt(halted),
        .pc(pc)
    );

    assign pc_out = pc;

    wire [11:0] instruction;

    instruction_memory IM(
        .address(pc),
        .instruction(instruction)
    );

    assign instruction_out = instruction;

    wire [3:0] opcode;
    wire [1:0] rd;
    wire [1:0] rs;
    wire [3:0] immediate;

    assign opcode    = instruction[11:8];
    assign rd        = instruction[7:6];
    assign rs        = instruction[5:4];
    assign immediate = instruction[3:0];

    wire reg_write;
    wire mem_read;
```

```
    wire mem_write;
    wire alu_src;
    wire halt;
    wire [2:0] alu_sel;

    control_unit CU(
        .opcode(opcode),
        .reg_write(reg_write),
        .mem_read(mem_read),
        .mem_write(mem_write),
        .alu_src(alu_src),
        .halt(halt),
        .alu_sel(alu_sel)
    );

    assign halted = halt;

    wire [3:0] read_data1;
    wire [3:0] read_data2;
    wire [3:0] write_data;

    register_file RF(
        .clk(clk),
        .reset(reset),
        .reg_write(reg_write && !halt),
        .read_addr1(rs),
        .read_addr2(rs),
        .write_addr(rd),
        .write_data(write_data),
        .read_data1(read_data1),
        .read_data2(read_data2)
    );

    wire [3:0] alu_result;
    wire zero;
    wire carry;

    alu ALU(
        .A(read_data1),
        .B(immediate),
        .ALU_Sel(alu_sel),
        .Result(alu_result),
        .Zero(zero),
        .Carry(carry)
    );
```

```
    wire [3:0] memory_data;

    data_memory DM(
        .clk(clk),
        .reset(reset),
        .mem_read(mem_read),
        .mem_write(mem_write),
        .address(immediate),
        .write_data(read_data1),
        .read_data(memory_data)
    );

    assign write_data =
        (opcode == 4'b0110) ? memory_data :
        (opcode == 4'b1000) ? read_data1 :
        alu_result;

endmodule
```

6.8 Module Hierarchy

```
processor (top level)
|-- program_counter      instance PC
|-- instruction_memory   instance IM
|-- control_unit         instance CU
|-- register_file        instance RF
|-- alu                  instance ALU
+-- data_memory          instance DM
```

6.9 Module Interfaces

Module	Inputs	Outputs
alu	A[3:0], B[3:0], ALU_Sel[2:0]	Result[3:0], Zero, Carry
register_file	clk, reset, reg_write, read_addr1[1:0], read_addr2[1:0], write_addr[1:0], write_data[3:0]	read_data1[3:0], read_data2[3:0]
control_unit	opcode[3:0]	reg_write, mem_read, mem_write, alu_src, halt, alu_sel[2:0]
instruction_memory	address[3:0]	instruction[11:0]

Module	Inputs	Outputs
data_memory	clk, reset, mem_read, mem_write, address[3:0], write_data[3:0]	read_data[3:0]
program_counter	clk, reset, enable, halt	pc[3:0]
processor	clk, reset	pc_out[3:0], instruction_out[11:0], halted

6.10 Design Notes

- The enable input of the program counter is tied to 1, so the counter runs until halt is asserted.
- Both read addresses of the register file are connected to rs, so the second read port (read_data2) is not used in this version.
- The register-write enable is reg_write AND NOT halt, so no register is changed while the processor is halted.
- The write-back mux is a conditional assignment: opcode 0110 (LOAD) selects the memory data, opcode 1000 (MOV) selects read_data1, and every other opcode selects the ALU result.
- pc_out and instruction_out are brought out to the top level only so that the testbench can monitor them.

7. Testbench

The testbench generates a clock with a 10 ns period, holds reset high for the first 10 ns and then releases it. It prints the program counter, the instruction and the halted flag with `$monitor`, waits until the processor halts, and ends the simulation 10 ns later. A safety timeout ends the run at 200 ns if the processor never halts. The VCD file processor.vcd is written for waveform viewing.

```
module processor_tb;

    reg clk;
    reg reset;

    wire [3:0] pc_out;
    wire [11:0] instruction_out;
    wire halted;

    // Processor connection
```

```
    processor uut(
        .clk(clk),
        .reset(reset),
        .pc_out(pc_out),
        .instruction_out(instruction_out),
        .halted(halted)
    );

    // Clock generation
    always #5 clk = ~clk;

    initial begin

        $dumpfile("processor.vcd");
        $dumpvars(0, processor_tb);

        clk = 0;
        reset = 1;

        $display("4-BIT PROCESSOR SIMULATION");
        $display("--------------------------");

        #10;
        reset = 0;

        // Display processor execution
        $monitor("Time=%0t | PC=%d | Instruction=%b | HALT=%b",
                 $time, pc_out, instruction_out, halted);

        // Wait until processor executes HALT
        wait(halted == 1'b1);

        #10;

        $display("--------------------------");
        $display("PROCESSOR HALTED SUCCESSFULLY");
        $display("FINAL PC = %d", pc_out);
        $display("PROCESSOR TEST COMPLETED");

        $finish;

    end

    // Safety timeout
    initial begin
        #200;
```

```
        $display("Simulation timeout");
        $finish;
    end

endmodule
```

7.1 Testbench Operation

Time (ns)	Event
0	clk = 0 and reset = 1; the VCD dump starts and the two heading lines are printed
5	First rising clock edge; reset is still high, so the program counter stays at 0
10	reset = 0; the monitor starts printing PC, instruction and halted
15, 25, 35, 45	Rising edges at which the four instructions before HALT are committed
45	halted = 1, because the instruction at address 4 is HALT
55	After a 10 ns delay the final messages are printed and the simulation finishes
200	Safety timeout, never reached in this run

7.2 Monitoring and Termination

The `$monitor` task prints a line whenever the PC, the instruction or halted changes, which is why one line appears at every program-counter step. The `wait(halted == 1'b1)` statement suspends the test until the processor reports that HALT has been decoded. A second `initial` block acts as a watchdog: if the processor never halted, for example because of a coding error, the simulation would still finish at 200 ns instead of running forever.

8. Test Program and Results

8.1 Test Program

The test program in the instruction memory has five instructions. All registers start at 0 after reset.

Addr	Machine code	Opcode	rd	rs	Immediate	Instruction	Operation
0	0000 0001 0010	0000	01	00	0010	ADD R1, R0, 2	R1 ← 0 + 2
1	0001 0011 0100	0001	00	11	0100	SUB R0, R3, 4	R0 ← 0 − 4
2	0010 0101 0110	0010	01	01	0110	AND R1, R1, 6	R1 ← R1 AND 6
3	0011 0111 0000	0011	01	11	0000	OR R1, R3, 0	R1 ← R3 OR 0
4	1111 0000 0000	1111	–	–	–	HALT	Stop

Tracing the program by hand gives R1 = 0010 after ADD, R0 = 1100 after SUB (0 − 4 wraps around in 4 bits to 12), R1 = 0010 after AND, and R1 = 0000 after OR. The final register contents are therefore R0 = 1100, R1 = 0000, R2 = 0000 and R3 = 0000. The testbench does not print register values, so these figures come from the program and the design, not from the console output.

Datapath values for each instruction

Addr	Instruction	alu_sel	ALU input A	ALU input B	ALU result	Zero	Written to
0	ADD R1, R0, 2	000	R0 = 0000	0010	0010	0	R1
1	SUB R0, R3, 4	001	R3 = 0000	0100	1100	0	R0
2	AND R1, R1, 6	010	R1 = 0010	0110	0010	0	R1
3	OR R1, R3, 0	011	R3 = 0000	0000	0000	1	R1
4	HALT	–	–	–	–	–	none

Register contents after each instruction (hand-traced)

After address	R0	R1	R2	R3
Reset	0000	0000	0000	0000
0 (ADD)	0000	0010	0000	0000
1 (SUB)	1100	0010	0000	0000
2 (AND)	1100	0010	0000	0000
3 (OR)	1100	0000	0000	0000

The Carry output is 0 for all four instructions, since only an addition with a carry-out sets it.

8.2 Console Output

The log below is the output of the simulation on EDA Playground.

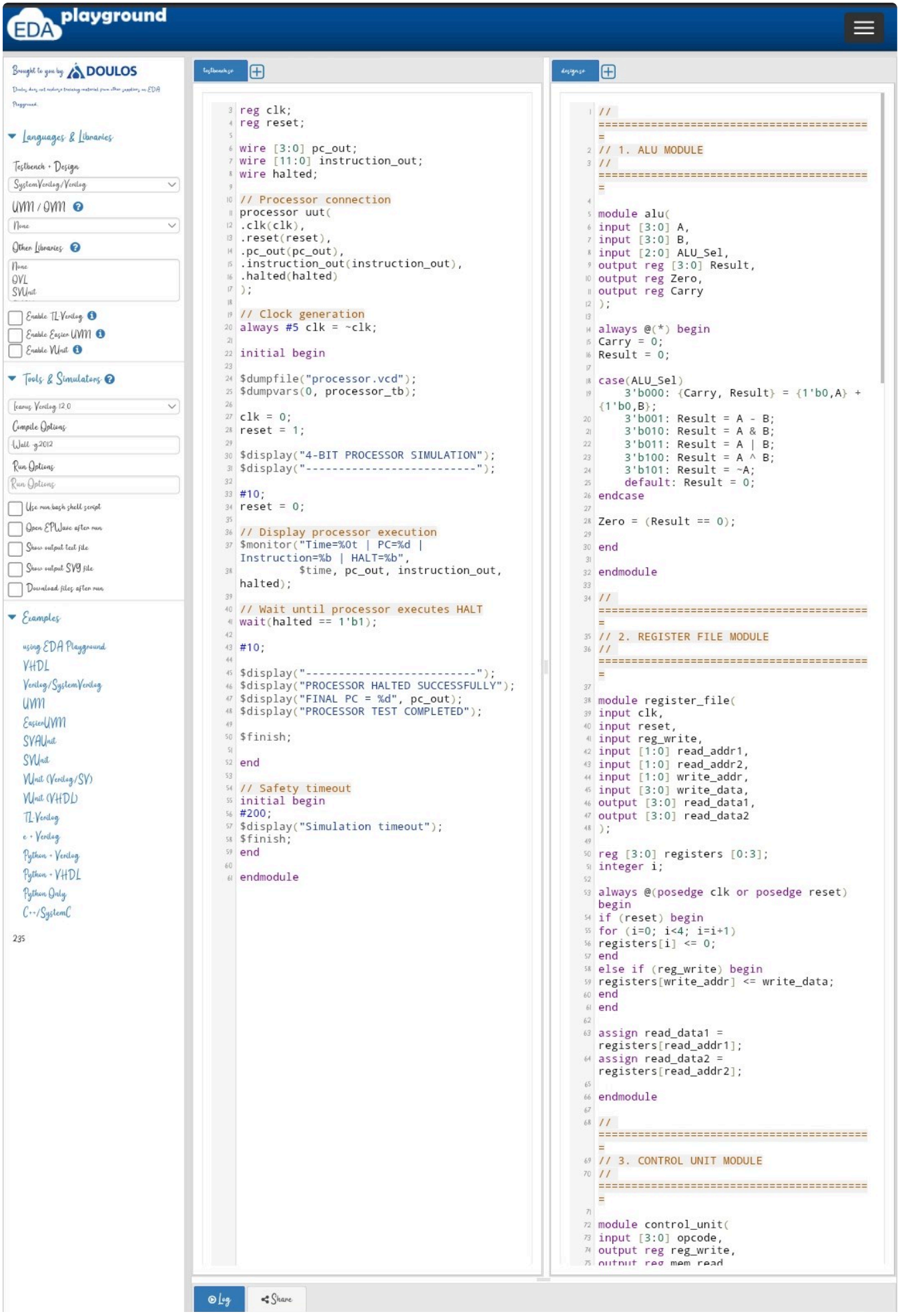
```
[2026-11-03 15:35:24 UTC] iverilog '-Wall' '-g2012' design.sv testbench.sv
&& unbuffer vvp a.out
design.sv:178: warning: @* is sensitive to all 16 words in array 'memory'.
design.sv:215: warning: @* is sensitive to all 16 words in array 'memory'.
VCD info: dumpfile processor.vcd opened for output.
4-BIT PROCESSOR SIMULATION
--------------------------
Time=10 | PC= 0 | Instruction=000000010010 | HALT=0
Time=15 | PC= 1 | Instruction=000100110100 | HALT=0
Time=25 | PC= 2 | Instruction=001001010110 | HALT=0
Time=35 | PC= 3 | Instruction=001101110000 | HALT=0
Time=45 | PC= 4 | Instruction=111100000000 | HALT=1
--------------------------
PROCESSOR HALTED SUCCESSFULLY
FINAL PC =  4
PROCESSOR TEST COMPLETED
testbench.sv:50: $finish called at 55 (1s)
```

Time (ns)	PC	Instruction	Meaning
10	0	0000 0001 0010	Reset released; ADD R1, R0, 2 is on the instruction bus
15	1	0001 0011 0100	First rising edge after reset: ADD completes and PC moves on
25	2	0010 0101 0110	SUB completes; AND is fetched

Time (ns)	PC	Instruction	Meaning
35	3	0011 0111 0000	AND completes; OR is fetched
45	4	1111 0000 0000	OR completes; HALT is fetched and halted goes high
55	4	–	Simulation ends 10 ns after halt; the 200 ns timeout is never reached

8.3 Simulation Screenshot

Figure 4 is the screenshot of the same run, showing the testbench, the design and the console output together.




```systemverilog
reg clk;
reg reset;

wire [3:0] pc_out;
wire [11:0] instruction_out;
wire halted;

// Processor connection
processor uut(
.clk(clk),
.reset(reset),
.pc_out(pc_out),
.instruction_out(instruction_out),
.halted(halted)
);

// Clock generation
always #5 clk = ~clk;

initial begin

$dumpfile("processor.vcd");
$dumpvars(0, processor_tb);

clk = 0;
reset = 1;

$display("4-BIT PROCESSOR SIMULATION");
$display("--------------------------");

#10;
reset = 0;

// Display processor execution
$monitor("Time=%0t | PC=%d |
Instruction=%b | HALT=%b",
         $time, pc_out, instruction_out,
halted);

// Wait until processor executes HALT
wait(halted == 1'b1);

#10;

$display("--------------------------");
$display("PROCESSOR HALTED SUCCESSFULLY");
$display("FINAL PC = %d", pc_out);
$display("PROCESSOR TEST COMPLETED");

$finish;

end

// Safety timeout
initial begin
#200;
$display("Simulation timeout");
$finish;
end

endmodule
```

```systemverilog
//
==========================================
=
// 1. ALU MODULE
//
==========================================
=

module alu(
input [3:0] A,
input [3:0] B,
input [2:0] ALU_Sel,
output reg [3:0] Result,
output reg Zero,
output reg Carry
);

always @(*) begin
Carry = 0;
Result = 0;

case(ALU_Sel)
    3'b000: {Carry, Result} = {1'b0,A} +
{1'b0,B};
    3'b001: Result = A - B;
    3'b010: Result = A & B;
    3'b011: Result = A | B;
    3'b100: Result = A ^ B;
    3'b101: Result = ~A;
    default: Result = 0;
endcase

Zero = (Result == 0);

end

endmodule

//
==========================================
=
// 2. REGISTER FILE MODULE
//
==========================================
=

module register_file(
input clk,
input reset,
input reg_write,
input [1:0] read_addr1,
input [1:0] read_addr2,
input [1:0] write_addr,
input [3:0] write_data,
output [3:0] read_data1,
output [3:0] read_data2
);

reg [3:0] registers [0:3];
integer i;

always @(posedge clk or posedge reset)
begin
if (reset) begin
for (i=0; i<4; i=i+1)
registers[i] <= 0;
end
else if (reg_write) begin
registers[write_addr] <= write_data;
end
end

assign read_data1 =
registers[read_addr1];
assign read_data2 =
registers[read_addr2];

endmodule

//
==========================================
=
// 3. CONTROL UNIT MODULE
//
==========================================
=

module control_unit(
input [3:0] opcode,
output reg reg_write,
output reg mem_read
```

```
[2026-11-03 15:35:24 UTC] iverilog '-Wall' '-g2012' design.sv testbench.sv  && unbuffer vvp a.out
design.sv:178: warning: @* is sensitive to all 16 words in array 'memory'.
design.sv:215: warning: @* is sensitive to all 16 words in array 'memory'.
VCD info: dumpfile processor.vcd opened for output.
4-BIT PROCESSOR SIMULATION
--------------------------
Time=10 | PC= 0 | Instruction=000000010010 | HALT=0
Time=15 | PC= 1 | Instruction=000100110100 | HALT=0
Time=25 | PC= 2 | Instruction=001001010110 | HALT=0
Time=35 | PC= 3 | Instruction=001101110000 | HALT=0
Time=45 | PC= 4 | Instruction=111100000000 | HALT=1
--------------------------
PROCESSOR HALTED SUCCESSFULLY
FINAL PC =  4
PROCESSOR TEST COMPLETED
testbench.sv:50: $finish called at 55 (1s)
Done
```

Figure 4: Simulation of the processor on EDA Playground (testbench, design and console output)

8.4 Simulation Waveform

Figure 5 shows the waveform of the same run, viewed in EPWave from the VCD file. Reset is high until 10 ns. After it is released, the program counter steps to 1, 2, 3 and 4 at 15, 25, 35 and 45 ns, and the instruction bus changes in step (hexadecimal 012, 134, 256, 370 and F00). The halted signal goes high at 45 ns when the HALT instruction appears, and the program counter stays at 4. The waveform agrees with the console output.

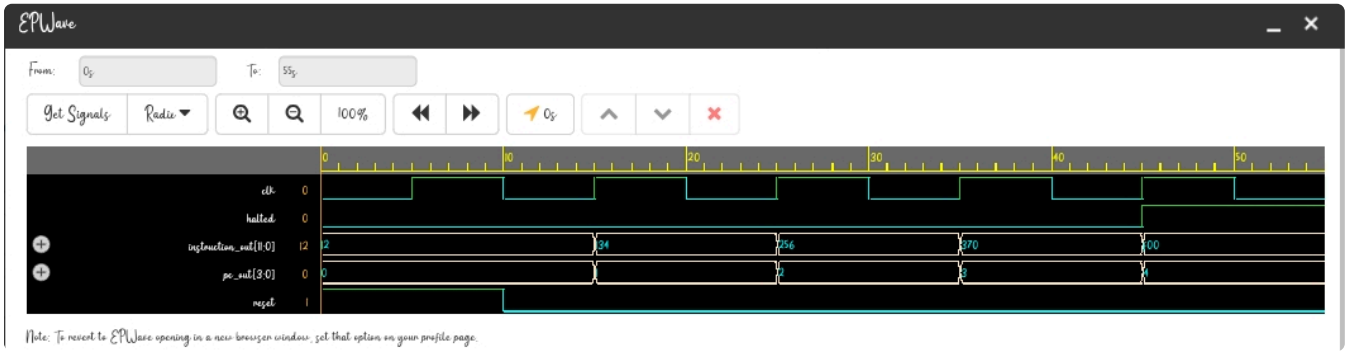


Figure 5: Simulation waveform of clk, halted, instruction_out[11:0], pc_out[3:0] and reset (0 to 55 ns)

8.5 Verification Summary

Feature	Evidence	Status
Reset sets PC to 0	Console and waveform show PC = 0 at 10 ns	Observed
Sequential fetch	PC and instruction bus step through addresses 0 to 4	Observed
Instruction memory contents	The five words on the bus match the memory initialisation	Observed

Feature	Evidence	Status
HALT stops the processor	halted = 1 and PC stays at 4	Observed
ADD, SUB, AND, OR results	Hand trace of the design	Not printed by the testbench
XOR, NOT, LOAD, STORE, MOV	Not used by the test program	Not exercised

8.6 Proposed Extended Test Program

To exercise every implemented instruction and to print the register contents, the following program is proposed. It has not been simulated for this report; the expected values come from a hand trace of the design.

Addr	Instruction	Machine code	Expected effect
0	ADD R1, R0, 9	0000 0100 1001	R1 = 1001
1	ADD R2, R0, 5	0000 1000 0101	R2 = 0101
2	XOR R3, R1, 12	0100 1101 1100	R3 = 1001 XOR 1100 = 0101
3	NOT R0, R2	0101 0010 0000	R0 = 1010
4	STORE R1, 3	0111 0001 0011	DMEM[3] = 1001
5	LOAD R2, 3	0110 1000 0011	R2 = 1001
6	MOV R3, R0	1000 1100 0000	R3 = 1010
7	ADD R1, R1, 8	0000 0101 1000	R1 = 0001 (9 + 8 = 17 wraps to 1)
8	SUB R2, R2, 9	0001 1010 1001	R2 = 0000
9	HALT	1111 0000 0000	Processor stops

The memory initialisation in `instruction_memory` would then be:

```
memory[0] = 12'b000001001001;
memory[1] = 12'b000010000101;
memory[2] = 12'b010011011100;
memory[3] = 12'b010100100000;
memory[4] = 12'b011100010011;
memory[5] = 12'b011010000011;
memory[6] = 12'b100011000000;
memory[7] = 12'b000001011000;
memory[8] = 12'b000110101001;
memory[9] = 12'b111100000000;
```

The testbench can print the registers and the memory word after the halt with these lines:

```
$display("R0=%b R1=%b R2=%b R3=%b",
         uut.RF.registers[0], uut.RF.registers[1],
         uut.RF.registers[2], uut.RF.registers[3]);
$display("DMEM[3]=%b", uut.DM.memory[3]);
```

The expected final values are R0 = 1010, R1 = 0001, R2 = 0000, R3 = 1010 and DMEM[3] = 1001. The processor should halt at 95 ns, which is before the 200 ns timeout.

8.7 Discussion

The output shows the behaviour expected from a single-cycle processor. The program counter advances by one on each clock edge, every instruction word on the bus matches the contents of the instruction memory, and the processor stops with PC fixed at 4 as soon as HALT is decoded. The two compiler warnings come from the combinational read of the memory arrays in the instruction and data memories, where `@(*)` is sensitive to all 16 words; they do not affect the simulation result.

In the present datapath the ALU's second input is always the immediate field. The alu_src signal is generated by the control unit but is not yet used to select between inputs, and the Zero and Carry flags are produced but not used, because there is no branch instruction. These are natural points for extension.

Design observations

- The control unit generates alu_src, but the ALU's B input is connected directly to the immediate, so the signal has no effect yet.
- The Zero and Carry outputs of the ALU are not connected to any other block. They become useful when conditional branch instructions are added.
- Carry is produced only for addition. For subtraction the result simply wraps around modulo 16 and no borrow flag is generated.

- The data memory is cleared on reset and returns 0 whenever mem_read is low, so a LOAD from an address that was never written returns 0.
- Opcodes 1001 to 1110 are free and can be used for new instructions, such as jumps and branches, without changing the instruction width.

9. Advantages, Limitations and Applications

Advantages

- Simple modular structure with one Verilog module per block.
- Single-cycle execution: one instruction per clock.
- Register file, data memory and ALU make it a complete small processor.
- Fully simulated with a self-contained testbench.

Limitations

- 16 words of instruction memory and 16 words of data memory.
- No branch, jump or subroutine instructions.
- The test program has only four ALU instructions and a HALT; LOAD, STORE, XOR, NOT and MOV are implemented but not exercised by this testbench.
- Register contents are not printed by the testbench.

Applications

- Teaching processor design and Verilog HDL.
- Base design for FPGA experiments.
- Starting point for larger 8-bit or 16-bit processors.

10. Future Scope

- Extend the test program to cover LOAD, STORE, XOR, NOT and MOV, and print register contents in the testbench.
- Use the alu_src signal to select between register and immediate operands.
- Add jump and conditional branch instructions using the Zero and Carry flags.
- Increase the data width and memory size.
- Implement the design on an FPGA board and measure resource use and clock speed.
- Add pipelining for higher throughput.

11. Conclusion

A 4-bit single-cycle processor with a 12-bit instruction format was designed in Verilog HDL using seven modules: ALU, register file, control unit, instruction memory, data memory, program counter and the top-level processor. The testbench shows the program counter stepping through the five stored instructions and the processor halting at 45 ns with a final PC of 4. The project gives a clear picture of how instructions are fetched, decoded and executed at the RTL level and provides a base for more advanced designs.

Learning outcomes

- Understanding of the main parts of a processor and how they work together.
- Practice in writing synthesisable Verilog for combinational and sequential logic.
- Experience in designing an instruction format and a control unit.
- Skill in writing a testbench and in reading console output and waveforms to verify a design.

References

1. S. Palnitkar, *Verilog HDL: A Guide to Digital Design and Synthesis*, 2nd ed., Prentice Hall, 2003.
2. M. M. Mano and M. D. Ciletti, *Digital Design: With an Introduction to the Verilog HDL*, Pearson.
3. S. Brown and Z. Vranesic, *Fundamentals of Digital Logic with Verilog Design*, McGraw-Hill.
4. D. A. Patterson and J. L. Hennessy, *Computer Organization and Design: The Hardware/Software Interface*, Morgan Kaufmann.
5. IEEE Standard for Verilog Hardware Description Language, IEEE Std 1364-2005.